

Capitolul 3

Sisteme de ecuații

3.1 Eliminarea gaussiană

Metoda rezolvării sistemului liniar $Ax = b$ prin eliminare gaussiană este implementată în MATLAB de codul de mai jos.

Algorithm (Eliminarea gaussiană).

```
for  $j = 1, \dots, n - 1$ 
    if  $a_{jj} = 0$ , stop, end
    for  $i = j + 1, \dots, n$ 
        for  $k = j + 1, \dots, n$ 
             $a_{ik} = a_{ik} - \frac{a_{ij}}{a_{jj}} a_{jk}$ 
        end
         $b_i = b_i - \frac{a_{ij}}{a_{jj}} b_j$ 
    end
end
```

Algorithm (Substituția înapoi pentru eliminarea gaussiană).

```
for  $i = n, \dots, 1$ 
    for  $j = i + 1, \dots, n$ 
         $b_i = b_i - a_{ij} x_j$ 
    end
     $x_i = \frac{b_i}{a_{ii}}$ 
end
```

```
function [A,x]=gauss(A,b)
n=length(A);
for j=1:n-1
    if abs(A(j,j)) == 0
        error('zero pivot encountered!');
    end
    for i=j+1:n
        mult=A(i,j)/A(j,j);
        for k=j:n
            A(i,k)=A(i,k)-mult*A(j,k);
        end
        b(i)=b(i)-mult*b(j);
    end
end
end
```

```

for i=n:-1:1
    for j=i+1:n
        b(i)=b(i)-A(i,j)*x(j);
    end
    x(i)=b(i)/A(i,i);
end

```

Funcția `length` returnează lungimea celei mai mari dimensiuni a matricii argument, care, în cazul matricilor pătratice, este egală cu ordinul matricii. Aplicată unui vector, această funcție îi returnează lungimea.

După definirea unei matrici A și a unui vector b

```

>> A=[2 1 5; 4 4 -4; 1 3 1];
>> b=[9; -8; 4];

```

putem aplica eliminarea gaussiană pentru rezolvarea sistemului $Ax = b$:

```

>> [A,x]=gauss(A,b)

```

Această funcție va returna atât soluția x a sistemului, cât și noua formă a matricii A după eliminarea gaussiană.

3.2 Factorizarea $PA = LU$

Cea mai sofisticată variantă a eliminării gaussiene este factorizarea $PA=LU$. Comanda `lu` din MATLAB primește ca parametru o matrice pătratică A , și returnează matricile L , U și P . Exemplul următor definește o matrice A și îi calculează factorizarea:

```

>> A=[2 1 5; 4 4 -4; 1 3 1];
>> [L,U,P]=lu(A)

```

Pentru rezolvarea sistemelor de ecuații liniare, MATLAB dispune de operatorul `\`. Astfel, comanda

```

>> x=A\b

```

rezolvă sistemul liniar $Ax = b$ folosind factorizarea $PA=LU$.

3.3 Metode iterative

Metoda lui Jacobi pentru rezolvarea sistemului liniar $Ax = b$ poate fi implementată în MATLAB astfel:

Algorithm (Metoda lui Jacobi).

$$\begin{aligned}
 x_0 &= \text{vectorul inițial} \\
 x_{k+1} &= D^{-1}(b - (L + U)x_k) \text{ for } k = 0, 1, 2, \dots
 \end{aligned}$$

```
function x=jacobi(A,b,x0,k)
D=diag(diag(A));
L=tril(A)-D;
U=triu(A)-D;
x=x0;
for j=1:k
    x = inv(D)*(b-(L+U)*x);
end
```

Funcția `diag` returnează diagonală principală a matricii argument, sau o matrice având pe diagonală principală elementele vectorului argument. Funcția `tril` returnează partea de sub diagonală principală a matricii argument, inclusiv diagonală principală. Analog, funcția `triu` returnează partea de deasupra diagonalei principale a matricii argument, inclusiv diagonală principală. Funcția `inv` returnează inversa matricii argument.

Definind matricea A , vectorul b și vectorul inițial x_0 ca fiind format din zerouri:

```
>> A=[3 1 -1; 2 4 1; -1 2 5];
>> b=[-4; 4; 13];
>> x0=zeros(3,1);
```

putem aplica 10 pași ai metodei lui Jacobi pentru rezolvarea sistemului $Ax = b$ astfel:

```
>> x=jacobi(A,b,x0,10)
```

3.4 Metode pentru matrici simetrice și pozitiv definite

Verificarea simetriei unei matrici se poate face inspectând dacă matricea este egală cu transpusa ei:

```
>> A == A'
```

unde A' returnează transpusa matricii A . Dacă rezultatul returnat este o matrice de 1-uri, atunci matricea A este simetrică.

O matrice este pozitiv definită dacă și numai dacă toate valorile proprii ale sale sunt strict pozitive. Comanda `eig` returnează valorile proprii ale unei matrici. Astfel, dacă

```
>> eig(A)>0
```

returnează un vector de 1-uri, atunci matricea A este pozitiv definită.

Factorizarea Cholesky a unei matrici simetrice se realizează în MATLAB folosind comanda `chol`:

```
>> chol(A)
```

Comenzile de mai sus pot fi testate folosind matricea

```
>> A=[4 -2 2; -2 2 -4; 2 -4 11];
```

3.5 Sisteme de ecuații neliniare

Codul MATLAB pentru metoda lui Broyden II destinată rezolvării sistemului neliniar $F(x) = 0$ este:

Algorithm (Metoda lui Broyden II).

$x_0 =$ vectorul inițial

$B_0 =$ matricea inițială

for $i = 0, 1, 2, \dots$

$x_{i+1} = x_i - B_i F(x_i)$

$B_{i+1} = B_i + \frac{(\delta_{i+1} - B_i \Delta_{i+1}) \delta_{i+1}^T B_i}{\delta_{i+1}^T B_i \Delta_{i+1}}$

end

unde $\delta_{i+1} = x_{i+1} - x_i$ și $\Delta_{i+1} = F(x_{i+1}) - F(x_i)$.

```
function x=broyden2(F,x0,k)
n=length(x0);
B=eye(n,n);
for i=1:k
    x=x0-B*F(x0);
    delta=x-x0;
    Delta=F(x)-F(x0);
    B=B+(delta-B*Delta)*delta'*B/(delta'*B*Delta);
    x0=x;
end
```

De exemplu, putem defini funcția $F(u, v) = (v - u^3, u^2 + v^2 - 1)$ prin function handle-ul

```
>> F=@(x) [x(2)-x(1)^3;x(1)^2+x(2)^2-1];
```

Acum putem apela metoda lui Broyden II pentru rezolvarea sistemului neliniar $F(u, v) = 0$ ca mai jos:

```
>> x=broyden2(F,[1;1],10)
```

Comanda predefinită în MATLAB pentru rezolvarea sistemelor neliniare este **fsolve**, care primește ca argumente function handle-ul și valoarea lui x_0 :

```
>> x=fsolve(F,[1;1])
```

Probleme

1. Rezolvați sistemul

$$\begin{cases} -5x + y - 2z = 6 \\ x - 3y + z = -2 \\ 2x + y - 4z = -7 \end{cases}$$

folosind eliminarea gaussiană.

```
A =
-5.000000000000000    1.000000000000000   -2.000000000000000
         0   -2.800000000000000    0.600000000000000
         0         0   -4.500000000000000

x =
-1.539682539682540    0.523809523809524    1.111111111111111
```

2. Determinați factorizarea PA=LU a matricii $A = \begin{bmatrix} 3 & 1 & 2 \\ 6 & 3 & 4 \\ 3 & 1 & 5 \end{bmatrix}$.

$$\begin{aligned}
 L &= \begin{bmatrix} 1.0000000000000000 & 0 & 0 \\ 0.5000000000000000 & 1.0000000000000000 & 0 \\ 0.5000000000000000 & 1.0000000000000000 & 1.0000000000000000 \end{bmatrix} \\
 U &= \begin{bmatrix} 6.0000000000000000 & 3.0000000000000000 & 4.0000000000000000 \\ 0 & -0.5000000000000000 & 0 \\ 0 & 0 & 3.0000000000000000 \end{bmatrix} \\
 P &= \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

3. Folosiți metoda lui Jacobi pentru rezolvarea sistemului

$$\begin{cases} 3u - v + w = -2 \\ u - 8v - 2w = 1 \\ u + v + 5w = 4 \end{cases}$$

$$\begin{aligned}
 \mathbf{x} &= \\
 &-1.244351137996721 \\
 &-0.571066613176762 \\
 &1.162871124413259
 \end{aligned}$$

4. Implementați metoda Gauss-Seidel pentru rezolvarea sistemului $Ax = b$. Funcția MATLAB trebuie să aibă semnătura

`function x=gauss_seidel(A,b,x0,k)`

unde $\mathbf{x}_0$ este vectorul inițial, iar k este numărul de pași. Indicație: utilizați scrierea

$$(L + D)x_{k+1} = b - Ux_k,$$

a algoritmului.

Algoritm (Metoda Gauss–Seidel).

$$\begin{aligned}
 x_0 &= \text{vectorul inițial} \\
 x_{k+1} &= D^{-1}(b - Ux_k - Lx_{k+1}) \text{ for } k = 0, 1, 2, \dots
 \end{aligned}$$

5. Scrieți o funcție MATLAB care implementază metoda supra-relaxărilor succesive pentru rezolvarea sistemului $Ax = b$, având semnătura:

`function x=sor(A,b,x0,omega,k)`

unde $\mathbf{x}_0$ este vectorul inițial, ω este parametrul ω din cadrul algoritmului, iar k este numărul de pași.

Algoritm (Metoda supra-relaxărilor succesive (SRS)).

$$\begin{aligned} x_0 &= \text{vectorul inițial} \\ x_{k+1} &= (\omega L + D)^{-1}[(1 - \omega)Dx_k - \omega Ux_k] + \omega(D + \omega L)^{-1}b \text{ for } k = 0, 1, 2, \dots \end{aligned}$$

6. Arătați că matricea $A = \begin{bmatrix} 4 & -2 & 3 \\ -2 & 2 & -1 \\ 3 & -1 & 5 \end{bmatrix}$ este simetrică și pozitiv definită.

7. Determinați factorizarea Cholesky a matricii $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{bmatrix}$.

$$\begin{aligned} \text{ans} &= \\ &\begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

8. Implementați în MATLAB metoda gradientilor conjugați pentru găsirea soluției x a sistemului $Ax = b$, unde A este o matrice simetrică și pozitiv definită. Semnătura funcției trebuie să fie:

`function x=conjgrad(A,b,x0,k)`

unde x_0 este vectorul inițial și k reprezintă numărul de pași.

Algoritm (Metoda gradientilor conjugați).

$x_0 = \text{vectorul inițial}$

$d_0 = r_0 = b - Ax_0$

for $k = 0, 1, 2, \dots, n - 1$

if $r_k = 0$, **stop**, **end**

$$\alpha_k = \frac{r_k^T r_k}{d_k^T A d_k}$$

$$x_{k+1} = x_k + \alpha_k d_k$$

$$r_{k+1} = r_k - \alpha_k A d_k$$

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$$

$$d_{k+1} = r_{k+1} + \beta_k d_k$$

end

9. Aplicați metoda Gauss-Seidel și metoda supra-relaxărilor succesive cu $\omega = 1.2$ pentru a rezolva sistemele liniare de la Problemele 1 și 3. Aplicați metoda gradientilor conjugați pentru a rezolva sistemul $Ax = b$, unde matricea A este cea de la Problema 6, iar $b = \begin{bmatrix} 1 \\ -2 \\ 4 \end{bmatrix}$. Comparați rezultatele obținute cu cele date de operatorul `\` din MATLAB.

```

x =
-1.539683200989964
 0.523809710487128
 1.111110827126800
x =
-1.543948718553361
 0.524525140521467
 1.107669081510644
x =
-1.539682539682540
 0.523809523809524
 1.111111111111111
x =
-1.244896236737709
-0.571427837991276
 1.163264814945797
x =
-1.244894272956530
-0.571430691461751
 1.163264086807967
x =
-1.244897959183674
-0.571428571428572
 1.163265306122449
x =
-2.100000000000000
-2.300000000000000
 1.600000000000000
x =
-2.100000000000000
-2.300000000000000
 1.600000000000000

```

10. Scrieți o funcție în MATLAB care rezolvă sistemul neliniar $F(x) = 0$ prin metoda lui Newton, având următoarea semnătură

```
function x=newton(F,DF,x0,k)
```

unde F este function handle-ul, DF este function handle-ul Jacobianului lui F , x_0 este vectorul inițial și k reprezintă numărul de pași.

Algoritm (Metoda lui Newton pentru mai multe variabile).

$$\begin{aligned}
 x_0 &= \text{vectorul inițial} \\
 x_{k+1} &= x_k - (DF(x_k))^{-1}F(x_k) \text{ for } k = 0, 1, 2, \dots
 \end{aligned}$$

11. Scrieți o funcție în MATLAB care rezolvă sistemul neliniar $F(x) = 0$ prin metoda lui Broyden I, având următoarea semnătură

```
function x=broyden1(F,x0,k)
```

unde F este function handle-ul, x_0 este vectorul inițial și k reprezintă numărul de pași.

Algoritm (Metoda lui Broyden I). $x_0 =$ vectorul inițial $A_0 =$ matricea inițială**for** $i = 0, 1, 2, \dots$

$$x_{i+1} = x_i - A_i^{-1} F(x_i)$$

$$A_{i+1} = A_i + \frac{(\Delta_{i+1} - A_i \delta_{i+1}) \delta_{i+1}^T}{\delta_{i+1}^T \delta_{i+1}}.$$

endunde $\delta_{i+1} = x_{i+1} - x_i$ și $\Delta_{i+1} = F(x_{i+1}) - F(x_i)$.

12. Rezolvați sistemul neliniar $F(u, v) = 0$, unde $F(u, v) = (u^2 + 4v^2 - 4, 4u^2 + v^2 - 4)$ folosind metoda lui Broyden II cu vectorul inițial $(1, 1)$.

```
x =
    0.894427190999916
    0.894427190999916
```

13. Aplicați metoda lui Newton și metoda lui Broyden I cu vectorul inițial $(1, 1)$ pentru rezolvarea sistemului neliniar de la Problema 12. Comparați rezultatul cu cel dat de funcția predefinită `fsolve`.

```
x =
    0.894427190999916
    0.894427190999916
x =
    0.894427190999916
    0.894427190999916
x =
    0.894427191166385
    0.894427191166385
```